Ausgabe am **23.06.2025**
Abgabe bis zum **07.07.2025, 13:30 Uhr**

Hochschule Fulda
University of Applied Sciences

Prof. Dr. Thomas Wiemann

# Robotik
# Übungsblatt 6

Sommersemester 2024

In dieser Übung betrachten machen wir uns mit Lokalisierungs- und Pfadplanungsalgorithmen vertraut.

**Aufgabe A6.1 (Monte Carlo Lokalisierung - Theorie)** **(25 Punkte)**

(a) Beschreiben Sie die Idee der Monte-Carlo-Lokalisierung (MCL). Aus welchen Teilschritten besteht MCL?

(b) Wozu dient dabei der *Resampling*-Schritt?

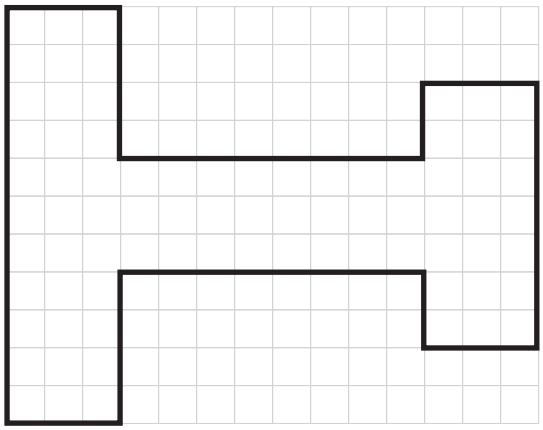
(c) Ist die Zahl der Partikel konstant oder variabel? Begründen Sie Ihre Antwort.

(d) Spielen Sie nun die einzelnen Schritte der Monte-Carlo-Lokalisierung für die folgende Abfolge von Aktionen in der unten gezeigten Umgebung mit den vorgegebenen Messergebnissen durch. Die Karten sind kontinuierlich, die Raster dienen nur als Hilfe für die Roboterbewegung.

Zeichnen Sie für jede Aktion jeweils eine plausible Partikelverteilung ein. Geben Sie zu jeder Zeichnung eine kurze Erklärung.

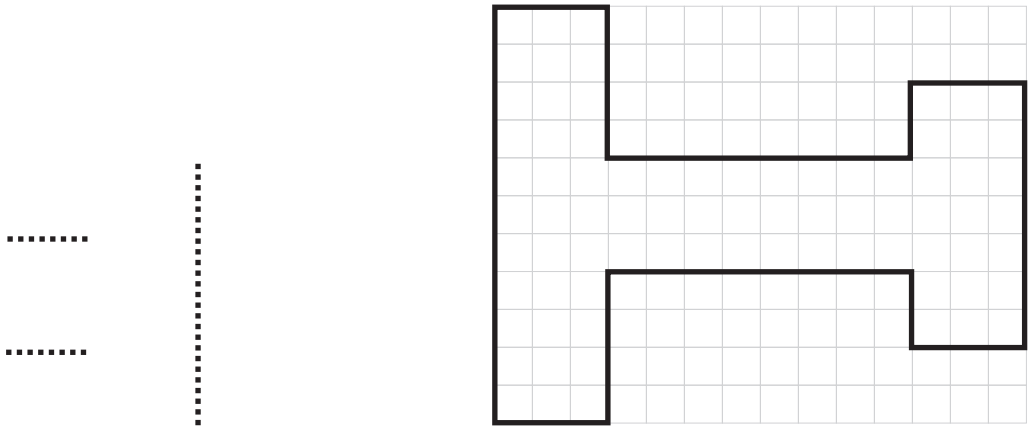
---

Initialisierung

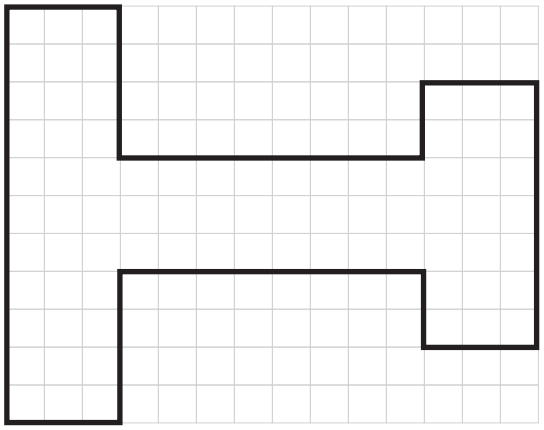


---

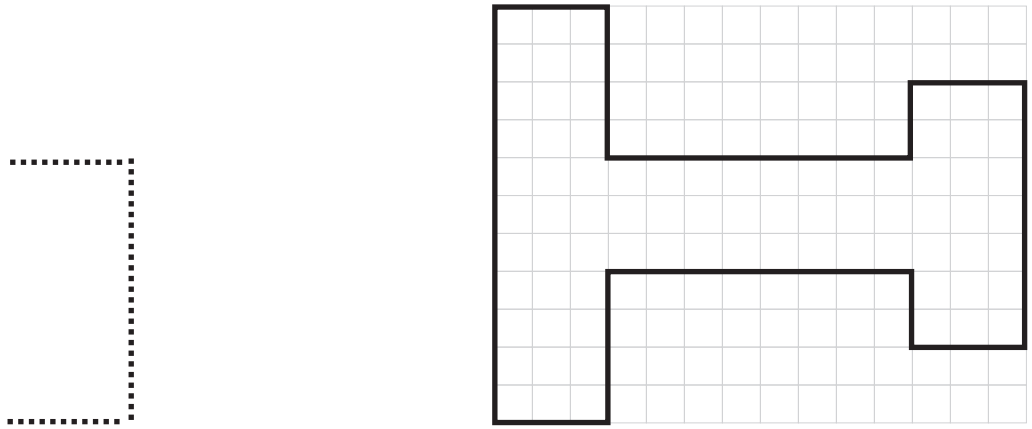
Erste Messung (Messergebnis links)



Bewegung um 2 Einheiten nach vorne



Zweite Messung (Messergebnis links)



**Aufgabe A6.2 (Monte-Carlo-Lokalisierung - Praxis)** **(25 Punkte)**

In dieser Aufgabe werden Sie einen Scan gegeben einer Pose in einer Karte simulieren. Der Nutzer soll eine Pose in der Karte einzeichnen, von dem aus ein LaserScan simuliert werden soll. Der resultierende LaserScan soll danach in RViz im Frame `map` sichtbar sein. Schreiben Sie dazu Node namens `scan_simulator_test` im Package `mcl_helper`. Das in Moodle vorgegebene Programmgerüst stellt die folgenden Publisher und Subscriber zur Verfügung:

Zur Bereitstellung der Karte auf dem Topic `/map` müssen Sie einen `map_server` starten, der die im Verzeichnis `maps` vorgegebene Karte publisht. Dazu gehen Sie wie folgt vor:

(a) Starten Sie den Map-Server mit passensen Parametern. Dazu wird Ihnen die Konfigurationsdatei `map_server_params` zur Verfügung gestellt. Starten Sie damit den Map-Server aus dem Stammverzeichnis des Pakets `mcl_helper`:

```
ros2 run nav2_map_server map_server --ros-args --params-file map_server_params.yaml
```

(b) Der Map-Server ist als sogenannte *Managed Node* implementiert. Diese erlauben es dem ROS2-System ihren Zustand zu melden, um sicherzustellen, dass das System erst dann started, wenn alle benötigten Nodes in einem gültigen Zustand sind. Entsprechend müssen wir den Map-Server zunächst konfigurieren und aktivieren, bevor wir ihn nutzen können:

```
ros2 lifecycle set /map_server configure
ros2 lifecycle set /map_server activate
```

(c) Jetzt sollte der Map-Server laufen und auf Anfragen reagieren. Starten Sie nun `RViz2` und fügen Sie einen Visualisier für das Topic `/map` vom Typ `map` hinzu. Wir wollen alle weiteren Daten im Map-Frame visualisieren, also setzen Sie den `Fixed Frame` in den `Global Options` auf `map`.

(d) Zuletzt müssen wir sicherstellen, das die aktuelle Karte auf dem Topic vom Map-Server gepublished wird. Da die Kartendaten potenziell groß sein können, werden dort neue Karten nur veröffentlicht, wenn ein Request gestellt wird. Sollte RViz2 noch nicht gelaufen sein, wenn Sie den Map-Server gestartet haben, wird das Programm noch keine Daten auf dem `/map`-Topic empfangen haben. Setzen Sie im dem Fall einen Request zum Neuladen der Karte in einem separaten Terminal ab:

```
ros2 service call /map_server/load_map
    nav2_msgs/srv/LoadMap "{map_url: maps/avz_6_floor_gazebo.yaml}"
```

Jetzt sollten Sie die Karten in RViz2 gerendert sehen.

Übersetzen Sie nun mit `colcon build` das vorgebene Programmgerüst. Starten Sie die Node `mcl_helper_test` aus dem Paket `mcl_helper`. Diese subscribed sich zum Topic `/map` des Map-Servers und dem Topic `goal_pose`, das von RViz2 gepublisht wird. Damit können Sie eine Pose aus RViz2 heraus senden. Klicken Sie dazu auf den Button *2D Goal Pose*. Dann können Sie einen Pfeil, der eine Position und Orientierung repräsentiert, in der GUI setzen. Diese Daten werden dann als Message vom Typ `geometry_msgs/msg/PoseStamped` versendet. Wenn die Node läuft, sollten Sie dann beim Absetzen einer *2D Goal Pose* von der `mcl_helper_test`-Node die Ausgabe *"Got Pose"* bekommen. Wenn Sie wie oben unter Schritt (d) beschrieben die Karte neu laden, sollte die Ausgabe *"Got Map"* erscheinen. Hinweis: Manchmam wird durch den Request beim ersten Aufruf keine neue Karte auf `/map` veröffentlicht. Setzen Sie in dem Fall den Request mehrfach ab, bis sie die Bestätigung von der Test-Node erhalten. Wenn Sie beide Nachrichten erzeugen können, haben Sie das System korrekt initialisiert und können jetzt die geforderte Funktionalität implementieren.

(a) Fügen Sie der Klasse `mcl_helper_test` eine Instanz der Klasse `mcl_helper::ScanSimulator` hinzu. Mit Hilfe dieser Klasse können Sie Laserscans in 2D-Gridmaps an gegebenen Posen simulieren.

(b) Sobald Sie eine neue Karte erhalten, fügen Sie diese mittels `setMap` in den Simulator ein.

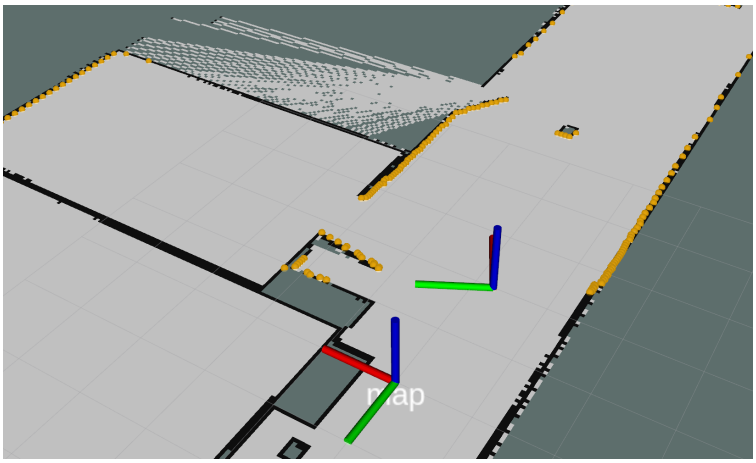
(c) Sobald Sie eine Pose auf dem Topic `goal_pose` bekommen, simulieren Sie einen Scan mittels der Methode `simulateScan` des Scan-Simulators. Diese schreibt die simulierten Daten automatisch in die Daten-Felder der übergebenenen `sensor_msgs::msg::LaserScan`. Damit der Simulator weiß, mit welchen Sensor-Parametern er die Ergebnisse berechnen soll, müssen Sie zunächst die Scanner-Parameter in der übergebenen `sensor_msgs::msg::LaserScan` Struktur setzen. Diese werden bei der Simulation zunächst ausgelesen, bevor die synthetischen Scan-Daten berechnet werden. Geeignete Werte sind:

```
angle_min = -1.5708
angle_max = +1.5708
range_min = 0;
range_max = 10
angle_increment = 0.0174553
```

Nach der Simulation veröffentlichen Sie die Nachricht im Frame `scan_simulator` auf dem Topic `/simulated_scan`.

(d) Die simulierten Daten werden im Frame `scan_simulator` berechnet. Damit sie korrekt in der Karte angezeigt werden, müssen wir noch die Transformationen zwischen `/map` und `/scan_simulator` bekannt geben. Dazu nutzen wir einen `tf2_ros::TransformBroadcaster` in der Klasse `mcl_helper_test`. Erzeugen sie Dazu eine Struktur des Typs `geometry_msgs::msg::TransformStamped` mit aktuellen Zeitstempel (Methode `now`) und `/map` als Frame und `/scan_simulator` als Child-ID. Füllen Sie die Transformation mit den Daten aus der empfangenen `geometry_msgs::msg::PoseStamped`.

Wenn Sie alles korrekt implementiert haben, sollten Sie bei einer Visualisierung des Topic `/simulated_scan` eine korrekte Korrespondenz mit der Karte sehen. Dies könnte z.B. so aussehen:




**Aufgabe B6.1 (Markow Lokalisierung)** **(25 Punkte)**

Gegeben sei die in Abbildung 1 abgebildete Welt eines mobilen Roboters. Der Roboter bewege sich deterministisch in einem Korridor mit 16 Gitterzellen. In einigen dieser Zellen sind Landmarken angebracht. Wenn der Roboter sich in einer der Zellen mit Landmarken befindet, so wird diese Landmarke von dem Roboter mit einer Wahrscheinlichkeit von 70% detektiert. In allen anderen Zellen (ohne Landmarken) meldet der Roboter aufgrund der Sensorungenaugikeit mit 25% Wahrscheinlichkeit dennoch eine Landmarke.

Der Roboter führt die folgenden 5 Aktionen aus:

1. Der Roboter detektiert eine Landmarke.
2. Der Roboter bewegt sich 2 Zellen im Uhrzeigersinn.
3. Der Roboter detektiert wieder eine Landmarke.
4. Der Roboter bewegt sich 4 Zellen im Uhrzeigersinn.
5. Der Roboter detektiert keine Landmarke.

Berechnen Sie die Aufenthaltswahrscheinlichkeit fur den Roboter für jede Zelle nach jedem einzelnen Schritt. In welcher Zelle ist demnach der Roboter vermutlich gestartet?

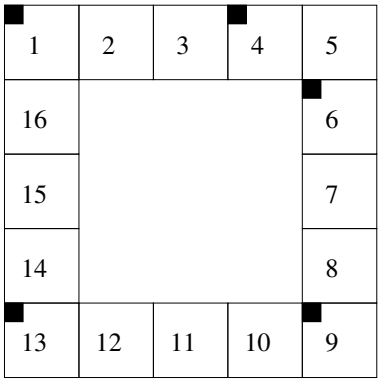
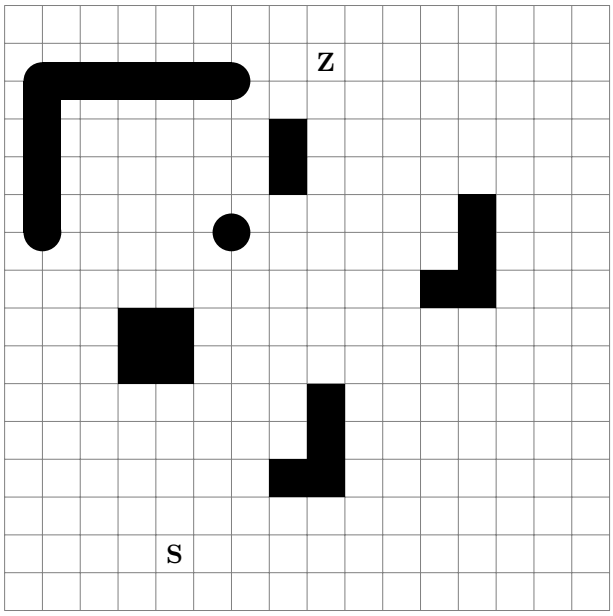



**Abbildung 1:** Markow-Lokalisierung

**Aufgabe B6.2 (Pfadplanung)** **(25 Punkte)**

Zur Navigation eines Roboters sei die folgende Rasterkarte zur 2D-Navigation eines Roboters gegeben. Der Roboter sei kreisförmig mit einem Durchmesser von 20cm und holonom in der Ebene, die räumliche Auflösung der Karte (Raster) sei 10cm.




(a) Überführen Sie die Karte in einen geeigneten Konfigurationsraum für den gegebenen Roboter.

(b) Transformieren Sie die Karte in eine Quadtree-Darstellung. Die kleinste Auflösung sei durch die Gitterzellen gegeben.

(c) Tragen Sie die Zellgewichte ein, die sich für eine Wavefront-Planung ausgehend vom Ziel ergeben. *Hinweis*: In einer Quadtree-Zerlegung hat nicht mehr jede Zelle eine uniforme Zahl von Nachbarzellen wie in einer uniformen Zerlegung! Berechnen Sie alle kürzesten Pfade vom Start $S$ zum Ziel $Z$ mit Hilfe des Wavefront-Algorithmus ohne Beachtung der Zellgröße und zeichnen Sie die Pfade in die Karte ein. Bewerten Sie alle Pfade hinsichtlich ihrer Befahrbarkeit. *Hinweis*: Sehr ähnliche Pfade können Sie zusammengefasst betrachten.

(d) Nehmen Sie nun an, ihre Umgebung sei zu einem gewissen Teil dynamisch, beispielsweise durch die Anwesenheit von Menschen. Eine zuvor berechnete Trajektorie kann also unter Umständen nicht vollständig abgefahren werden. Erläutern Sie eine Möglichkeit, dieses Problem zu kompensieren, ohne bei jedes Mal komplett neu planen zu müssen.